BLOCK 1

Advanced RAG & MCP

Part 1: Production RAG · Part 2: MCP Server

AGENDA

Block 1

RAGAS Baseline	Measure BEFORE touching any code. Four numbers to write down.
Fix 1 — Parent-child	Retrieve small, return large. Precision AND recall — not a trade-off.
Fix 2 — Hybrid search	BM25 (exact terms) + dense (semantic) + RRF (fusion).
Fix 3 — Reranking	Cross-encoder: joint query × document relevance scoring.
RAGAS Checkpoint	Before vs after. This is your report table.
MCP Architecture	What an MCP server is. 3 files. The stdio transport.
Build your server	3 tools. Docstrings for the LLM. Error handling.
Tests + Deliverable 1	MCP Inspector. 3 tests. Show the instructor.

01

Advanced RAG

Why your basic retriever fails — and the three fixes

DIAGNOSTIC

Measure first — do not touch anything before you have a number

Golden rule: you cannot improve what you do not measure.

RAGAS Metric	What it measures	Score < 0.6 means
context_recall	Are the relevant chunks being retrieved?	The retriever is missing important passages
context_precision	Are all retrieved chunks actually useful?	The retriever is returning noise
faithfulness	Does the answer only say what is in the context?	The agent is hallucinating
answer_relevancy	Does the answer actually address the question?	The agent is answering a different question

After building the pipeline, run 6b. Note your RAGAS scores

The five failure modes of a basic retriever

Component	What fails	RAGAS symptom	Fix
Chunking	Facts are cut at chunk boundaries — neither chunk is complete	context_recall low	Parent-child chunking
Embedding	Query (question) and document (answer) live in different embedding spaces	context_recall low	HyDE or query expansion
Retrieval	Dense search misses exact terms: acronyms, numbers, proper nouns	context_recall low	Hybrid: BM25 + dense + RRF
Re-ranking	Cosine similarity ≠ true relevance for this specific query	context_precision low	Cross-encoder
Context assembly	Too many chunks dilute the signal — the model loses key facts	faithfulness low	Threshold + dedup + max 5

Diagnostic guide: context_recall low → retrieval. context_precision low → reranking. faithfulness low → assembly or prompt.

Fix 1 — Parent-Child Chunking

Retrieve small (precision) · Return large (full context) — not a compromise, both at once.

The problem

With 150-word fixed chunks, a key fact can be cut in two.

Chunk A: '…21.5 million people are displaced every year'

Chunk B: 'by floods, droughts and rising sea levels…'

Neither A nor B contains the complete information.

→ context_recall low even though the data exists.

The solution

Two collections:

- Children (200 words) — indexed for retrieval

  → Small, precise, easy to match

- Parents (800 words) — returned to the LLM

  → Large, contain full context

Retrieve the child, return its parent.

RAGAS effect

↑ context_recall

Children match short queries better.

↑ context_precision

Only relevant parents are returned.

↑ faithfulness

The LLM has full context — less need to hallucinate.

Fix 2 — Hybrid Search: BM25 + Dense + RRF

Dense (your current setup)

Embedding vector similarity.

✓ Finds semantically related content even with different vocabulary.

e.g. 'coastal flooding' matches 'sea level rise'

✗ Misses exact terms.

e.g. 'UNHCR 2024' or '21.5 million' may not match — numbers and acronyms are underrepresented in embedding training.

BM25 — Sparse (add this)

Weighted term frequency.

✓ Finds exact keyword matches.

'UNHCR' always matches 'UNHCR'.

Numbers match numbers.

✗ Misses paraphrases.

'forced migration' does not match 'displacement'.

Typical gain: +10–20 pp context_recall on queries with proper nouns, numbers or acronyms.

RRF — Fusion

Reciprocal Rank Fusion:

score(doc) = Σ 1 / (60 + rank)

For each document: sum 1/(60+rank) across all lists.

No score normalisation needed.

Works directly on ranks.

Simple, robust, effective in practice.

Fix 3 — Cross-Encoder Reranking

Cosine similarity ≠ relevance. The cross-encoder reads the query AND the document together — it captures their interactions.

	Classic embedding	Cross-encoder
What it encodes	Query and document separately — no interaction	Query AND document together — full cross-attention
What it measures	Distance in vector space	Relevance of THIS document for THIS query
Speed	Fast — k vector comparisons	Slow — full inference per pair (query, doc)
Role in pipeline	Step 1: retrieve k=15 candidates	Step 2: rerank to top 5
RAGAS gain	Baseline	↑ context_precision: +15–25 pp typical

Full pipeline: hybrid retrieve (k=15 candidates) → cross-encoder rerank (top 5) → inject into LLM.

RAGAS Checkpoint — Document your progress

Re-run RAGAS on the same 5 questions. Fill in the After column. This table goes directly into your REPORT.md.

Metric	Baseline (before Block 1)	After parent-child	After hybrid search	After reranking
context_recall	___	___	___	___
context_precision	___	___	___	___
faithfulness	___	___	___	___
answer_relevancy	___	___	___	___

Targets: context_recall ≥ 0.70 and context_precision ≥ 0.70 after all three fixes.

02

MCP in Practice

Build your server · docstrings for the LLM · test

What an MCP server actually is

Concept	What it means	For your project
A Python script	A process that accepts JSON-RPC messages and returns results	You write one file: mcp_server.py
Tool definitions	JSON Schema for each tool — auto-generated from the function signature and docstring	The @mcp.tool() decorator does this for you
Transport	How the client and the server communicate	Use stdio for development — zero network config
Error handling	Every tool must return a string — never raise an exception	try/except around everything — a server crash = agent disconnection
Docstrings	The function docstring IS the tool description — the LLM reads it	Write it for the LLM, not for yourself

The docstring IS the tool description

The LLM reads the docstring to decide when to call the tool. This is the most important part of your MCP server.

①	**Use when…**	Explicit trigger. 'Use for facts after 2024.' Eliminates 80% of wrong-tool calls.
②	**Do NOT use for…**	Negative constraints matter as much as positive ones. 'Do NOT use for maths.' Without this, models overuse general-purpose tools.
③	**Returns…**	'Returns a list with title, URL and summary.' Tells the model whether this tool will give it what it needs next.
④	**Prefer X over Y when…**	'Prefer recall_memory before web_search if the topic was researched earlier in this session.' Explicit priority ordering.
⑤	**Concrete example**	e.g. query="UNHCR displacement 2024" — a concrete example dramatically improves the quality of arguments generated by the LLM.
⑥	**Precise naming**	'brave_web_search' not 'search'. 'read_local_file' not 'read'. Precise names reduce ambiguity when multiple tools overlap.

3-Tool Server Architecture

web_search

When to use: facts after 2024, citations, news.

Do NOT use for: maths, or topics already in memory.

Returns: title, URL, summary per result.

Mandatory error handling:

- Timeout → return error string
- HTTP error → return error string
- No results → return explanatory message

Never: raise an exception.

recall_memory

Use FIRST before web_search.

Avoids redundant API calls.

Useful filters:

- By source (UNHCR, World Bank)
- By date (recent data only)
- By topic

Returns: results with source attribution.

If nothing found → 'No relevant memories. Use web_search.'

store_finding

Use after web_search for verified results.

Do NOT store: speculation, unverified information.

Metadata to include:

- source (organisation name)
- url
- topic
- timestamp

Returns: storage confirmation.

Full cycle: web_search → verify → store_finding → recall_memory.

Testing your MCP server before connecting it to the agent

1	MCP Inspector (visual)	npx @modelcontextprotocol/inspector python mcp_server.py → opens a browser. Click through each tool manually. The fastest way to verify.
2	Test list_tools	Verify that all 3 tools appear with the correct names. If a tool is missing, the docstring or decorator is wrong.
3	Test web_search	Call with a real query from your domain. Verify the result is non-empty and correctly formatted.
4	Test store + recall	Store a finding → recall with the same query → verify the content comes back. This validates the complete memory cycle.

Testing your MCP server before connecting it to the agent

✓ Deliverable 1 — show the instructor before leaving

- ☐ RAGAS table: baseline vs improved (context_recall ≥ 0.65)
- ☐ MCP server: 3/3 tests passed
- ☐ Hybrid search and reranking present in your codebase